

DATA STRUCTURE

CS102

UNIT III — Tree

Non Linear Data Structure

BTechX

Non-Linear Data Structure

Key Characteristics

- ▶ Data elements are NOT placed sequentially or linearly
- ▶ Multiple paths exist for an element to reach another element
- ▶ Data elements are connected to one or more elements
- ▶ Examples: Tree, Graph

Tree Data Structure

What is a Tree?

- **Definition:** A hierarchical, non-linear data structure that stores data on multiple levels.
- **Root Node:** The topmost node of the tree — every tree has exactly one root.
- **Children:** Branches in the tree are known as children.
- **Structure:** Tree consists of nodes (circles) connected by edges (lines).

Why Use Trees?

Advantages of Tree Data Structures

- ▶ Enable efficient data searching and sorting
- ▶ Simple insertion and deletion operations
- ▶ Hierarchical nature represents relationships in a structured manner
- ▶ Used in file systems for quick data retrieval and updates

Applications of Tree Data Structure

Application	
D	Organizes files/folders in a hierarchical structure
e	Navigate and organize web pages efficiently
s	Model decision-making processes (e.g., weather forecast)
c	Represent arithmetic expressions with operators
r	Describes routing paths in networks
i	Quickly organizes and accesses large datasets
p	
t	
i	
o	
n	

Basic Terminologies

Node Types

- **Root:** The topmost node; has no parent. Every tree has exactly one root.
- **Node / Key:** Fundamental unit containing data and references to child nodes.

- **Parent Node:** A tree node with one or more child nodes.
- **Child Node:** Any node that has a parent. Node 'c' is child of 'p' if directly connected via an edge.
- **Siblings:** Nodes sharing the same parent are known as sibling nodes.
- **Leaf Node:** A node with no children. Also called External Node or Terminal Node.
- **Internal Node:** A node with at least one child (Non-Terminal Node). Root is internal if tree has more than one node.
- **Subtree:** A portion of a tree that can be viewed as a complete tree in itself.

Measurements

- **Edge:** Connecting link between two nodes. A tree with N nodes has at most N-1 edges.
- **Degree:** Total number of children a node has. The highest degree in a tree is the Degree of Tree.
- **Path:** A sequence of consecutive nodes and edges from source to destination.
- **Length of Path:** Total number of nodes in a path.
- **Level:** Count of edges from root to that node. Root node is at Level 1.
- **Height:** Total edges from a leaf node to a particular node in the longest path. Height of all leaf nodes = 0.
- **Depth:** Total edges from root to a particular node. Depth of root = 0.
- **Forest:** A collection of disjoint trees. Created by removing the root of a tree.

Level Rule: If node is at Level 'L', its children are at Level 'L+1'

Height of root node = Height of the tree

Types of Tree

Tree Types Overview

- ▶ Binary Tree — Each node has at most 2 children
- ▶ Binary Search Tree (BST) — Ordered binary tree for searching in $O(\log n)$
- ▶ AVL Tree — Self-balancing BST with balance factor -1, 0, or +1
- ▶ Heap Tree — Complete binary tree (Min-Heap / Max-Heap)

- ▶ General Tree — Nodes can have any number of children

Binary Tree

Definition

A Binary Tree is a type of tree data structure where each node can have a maximum of two child nodes — a left child node and a right child node.

Binary Tree Rules

- ▶ Each node can have either 0, 1, or 2 children
- ▶ Maximum degree of any node is at most two
- ▶ Node structure: Left Child Address | Data | Right Child Address

C Code — Node Structure

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
```

Types of Binary Tree

1. Strictly Binary Tree

Each non-terminal node must have both a non-empty left subtree and a non-empty right subtree.

- Example: Arithmetic expression trees with binary operators and operands, e.g., $(A+B)*C$

2. Complete Binary Tree

All levels are completely filled except possibly the last level, and the last level has all keys as left as possible.

- The leftmost side of the leaf node must always be filled first
- The last leaf node does not need to have a right sibling

At level L , maximum number of nodes = 2^L

3. Extended Binary Tree

Every null (missing) child is replaced by a special external node.

- Internal nodes → Original nodes with data (represented by circles)
- External nodes → Dummy nodes replacing NULL children (represented by squares)

4. Perfect Binary Tree

All internal nodes have exactly two children nodes. All leaf nodes are on the same level. All levels are completely filled.

5. Degenerate / Pathological Binary Tree

Each node contains only one child node — either on the left side or the right side of the tree.

Left Skewed Tree	
R	All children are RIGHT children
i	Looks like a linked list to the right
g	Maximum height for n nodes = $n-1$
h	
t	
S	
k	
e	
w	
e	
d	

T	
r	
e	
e	

6. Balanced Binary Tree

The difference between the height of the left and the right subtree for each node is either 0 or 1.

$$|\text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})| \leq 1 \quad \text{for every node}$$

Representation of a Binary Tree

Array Representation	
L	Each node has: Left ptr Data Right ptr
i	Root has no parent pointer (NULL)
n	Leaf nodes have both pointers = NULL
k	Flexible — no wasted space
e	Better for dynamic insertions
d	Uses more memory per node (2 pointers)
L	
i	
s	
t	
R	
e	
p	

r	
e	
s	
e	
n	
t	
a	
t	
i	
o	
n	

Tree Traversal

Tree Traversal is a way to visit each node exactly once in a systematic manner. A key application is representing arithmetic expressions.

Three Types of Tree Traversal

- ▶ Preorder — Root, Left Child, Right Child (Prefix notation)
- ▶ Inorder — Left Child, Root, Right Child (Infix notation)
- ▶ Postorder — Left Child, Right Child, Root (Postfix notation)

Preorder Traversal (Root → Left → Right)

Visit the root node first, then recursively traverse the left subtree, then the right subtree.

Example result: A B D E C F G

Inorder Traversal (Left → Root → Right)

Recursively traverse the left subtree first, then visit the root, then traverse the right subtree. For a BST, this gives sorted order.

Example result: D B E A F C G

Postorder Traversal (Left → Right → Root)

Recursively traverse the left subtree, then the right subtree, then visit the root.

Example result: D E B F G C A

Traversal C Code Templates

```
// Preorder
void printPreOrder(struct node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        printPreOrder(root->left);
        printPreOrder(root->right);
    }
}
```

Binary Expression Tree

An expression tree is a binary tree in which each internal node corresponds to an operator and each leaf node corresponds to an operand.

- Preorder traversal → Prefix expression (Polish notation)
- Inorder traversal → Infix expression (with parentheses)
- Postorder traversal → Postfix expression (Reverse Polish notation)

Example: (A+B) * (C+D) → Postorder: A B + C D + *

Binary Search Tree (BST)

Definition & Properties

A Binary Search Tree is a data structure that quickly allows us to maintain a sorted list of numbers.

BST Properties

- ▶ All nodes of LEFT subtree are LESS THAN the root node
- ▶ All nodes of RIGHT subtree are MORE THAN the root node
- ▶ Both subtrees of each node are also valid BSTs
- ▶ Search operation runs in $O(\log n)$ time

BST Insertion Algorithm

- ✓ Start at the root node
- ✓ Compare the value to insert with the current node's value
- ✓ If value is smaller → move to LEFT child
- ✓ If value is greater → move to RIGHT child
- ✓ Repeat until an empty spot is found
- ✓ Insert the new node at the empty spot

BST Deletion — Three Cases

Case	
A	Replace the leaf with NULL and free the space
c	Replace the node with its child; remove child from original position
t	Replace with inorder successor, remove successor from original position
i	
o	
n	

AVL Tree

Definition

AVL tree is a self-balancing Binary Search Tree in which each node maintains extra information called a balance factor, whose value is either -1, 0, or +1.

Named after inventors: Georgy Adelson-Velsky and Landis.

$$\text{Balance Factor} = \text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$$

AVL Tree Key Points

- ▶ Self-balancing property is maintained by the balance factor
- ▶ Controls height of BST by preventing skewed trees
- ▶ Rotation is performed when Balance Factor is other than -1, 0, or +1

Four Types of AVL Rotations

Rotation Type	
C	New node in LEFT subtree of LEFT subtree → Clockwise rotation (BF = 2)
o	New node in RIGHT subtree of RIGHT subtree → Anti-clockwise rotation (BF = -2)
n	New node in RIGHT subtree of LEFT subtree → RR on subtree + LL on full tree
d	New node in LEFT subtree of RIGHT subtree → LL on subtree + RR on full tree
i	
t	
i	
o	
n	
&	
A	
c	

t	
i	
o	
n	

LL Rotation (Clockwise)

When a node is inserted into the left subtree of the left subtree of A, making $BF = +2$. A single clockwise rotation is applied.

RR Rotation (Anti-clockwise)

When a node is inserted into the right subtree of the right subtree of A, making $BF = -2$. A single anti-clockwise rotation is applied.

LR Rotation (Double)

First apply RR rotation on the subtree, then apply LL rotation on the full tree (the first unbalanced ancestor node).

RL Rotation (Double)

First apply LL rotation on the subtree, then apply RR rotation on the full tree.

Practice Questions

Q1. Tree Terminologies

For the given tree (A as root, B and C as children of A, D and E as children of B, F and G as children of C, H as child of E, I and J as children of G), find:

- Siblings of Node D
- Degree of Node E
- Height of Tree
- Ancestors of Node H
- Descendants of Node C

- Level of Node G
- Depth of Tree
- Leaf Nodes
- Internal Nodes
- Left subtree of B

Q2. Tree Traversals

Find Preorder, Inorder, and Postorder traversals for the following trees:

- Tree 1: $A \rightarrow B (D, E \rightarrow G), C (F)$
- Tree 2: $F \rightarrow A (E, K \rightarrow C), D (H \rightarrow B, G)$
- Tree 3: $30 \rightarrow 20 (10 \rightarrow 15, 23, 25), 39 (35, 42)$

Q3. Construct BST

Construct a Binary Search Tree with the following elements in the given sequence:

45, 15, 79, 90, 10, 55, 12, 20, 50

Q4. AVL Tree Construction

Construct AVL trees for the following sequences, applying rotations as needed:

Exercise 1: 40, 20, 10, 25, 30, 22, 50

Exercise 2: 14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20

Exercise 3: 50, 20, 60, 10, 8, 15, 32, 46, 11, 48

Q5. Multiple Choice

How many rotations are required during the construction of an AVL tree if the following elements are added in the given sequence?

35, 50, 40, 25, 30, 60, 78, 20, 28

- 1. 2 left rotations, 2 right rotations
- 2. 2 left rotations, 3 right rotations
- 3. 3 left rotations, 2 right rotations

- 4. 3 left rotations, 1 right rotation

Construction of Binary Tree from Traversal

Given any two traversals (one of which must be Inorder), we can uniquely construct a binary tree.

From Preorder + Inorder

- First element of Preorder = Root
- Find Root in Inorder → splits into Left and Right subtrees
- Recursively repeat for each subtree

Preorder: A B D E C F		Inorder: D B E A F C
------------------------------	--	-----------------------------

From Postorder + Inorder

- Last element of Postorder = Root
- Find Root in Inorder → splits into Left and Right subtrees
- Recursively repeat for each subtree

Postorder: 9 15 7 20 3		Inorder: 9 3 15 20 7
-------------------------------	--	-----------------------------

References

1. <https://www.knowledgehut.com/blog/programming/types-of-trees-in-data-structure>
2. <https://www.codingninjas.com/studio/library/application-of-tree-in-data-structure>
3. <https://prepinsta.com/data-structures-algorithms/inorder-postorder-preorder-examples/>
4. <https://www.hackerearth.com/practice/algorithms/sorting/heap-sort/tutorial/>
5. <https://www.javatpoint.com/avl-tree>

— End of Unit V Notes —

BTechX